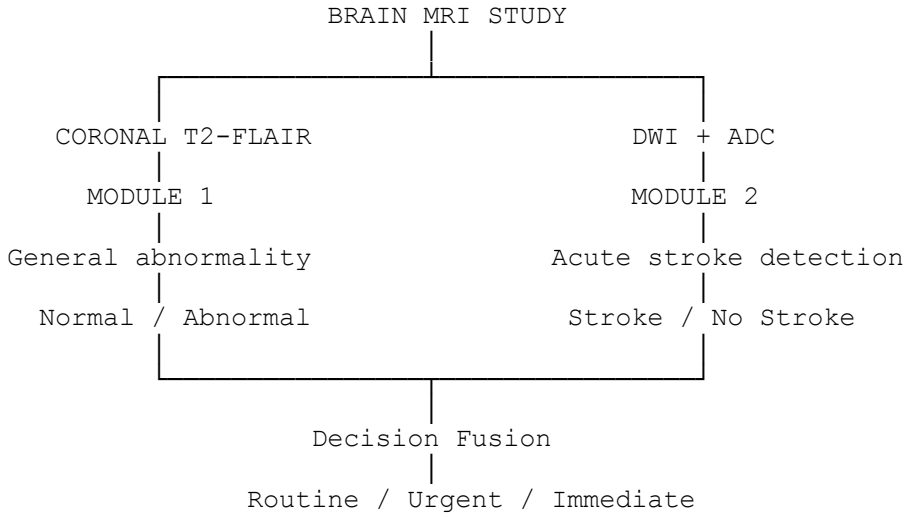
Project title

Multi-Sequence AI-Based Brain MRI Triage System Using Vision Foundation Models for General Abnormality Classification and Acute Ischemic Stroke Detection

The aim is not to automatically diagnose every neurological disease. The aim is to **triage brain MRI examinations before the radiologist reviews them**, so potentially important studies can be moved higher in the worklist.

The final system has two independently trained modules:



For now, Module 1 can be trained as **Normal vs Abnormal**. Borderline cases can be collected separately and introduced later as a third class once there are enough examples.

---

1. what a brain MRI actually looks like

An MRI examination is not one image.

A particular MRI **sequence** consists of a stack of 2D slices through the brain.

For example, a coronal FLAIR study might contain:

```
Patient A

Slice 01
Slice 02
Slice 03
...
Slice 25
```

while another patient might have only:

```
Patient B

Slice 01
...
Slice 21
```

This variation is normal.

The complete ordered stack represents the brain volume.

There are three common viewing planes:

```
AXIAL
Horizontal slices
looking from bottom → top

CORONAL
Front-to-back slices
like looking at the patient from the front

SAGITTAL
Side-to-side slices
```

For our project:

Module 1 uses **coronal T2-FLAIR** because that is the routine sequence available in the hospital dataset.

Module 2 will generally use **axial DWI and ADC** for acute ischemic stroke.

---

2. What is FLAIR?

FLAIR stands for:

Fluid-Attenuated Inversion Recovery.

It suppresses the bright signal from cerebrospinal fluid and makes many brain abnormalities easier to see.

It can reveal findings such as white-matter lesions, edema, infarcts, demyelination, tumors and various structural abnormalities.

Therefore Module 1 asks:

Does the patient's FLAIR study contain a convincing abnormality?

It initially outputs:

```
Normal
or
```

```
Abnormal
```

It is intentionally a broad screening task rather than trying to identify every possible disease.

---

3. What are DWI and ADC?

DWI stands for:

Diffusion-Weighted Imaging.

For acute ischemic stroke, a commonly used diffusion image is **b1000**, meaning:

b=1000 s/mm2b = 1000\ \text{s/mm}^2

A fresh infarct commonly appears:

```
DWI b1000 → bright
ADC       → low / dark
```

ADC stands for:

Apparent Diffusion Coefficient.

The ADC map helps confirm that the DWI brightness represents true restricted diffusion rather than another phenomenon.

Therefore our stroke module uses:

```
DWI + ADC
     ↓
Stroke / No Stroke
```

The **b1000 value is an MRI acquisition parameter associated with the DWI series**, not a class label in the Excel sheet.

---

4. What is DICOM?

Hospital MRI images are normally stored in **DICOM** format.

A DICOM file contains:

```
Image pixels
+
medical/image-acquisition metadata
```

Metadata can contain information such as:

```
Patient ID
Study ID
Series Description
Image Position
Image Orientation
Pixel Spacing
Slice Thickness
Scanner manufacturer
etc.
```

Therefore the original hospital dataset should remain in DICOM until it has been safely de-identified.

---

5. Initial data workflow

The current raw-data workflow is:

```
Hospital PACS export
        ↓
Raw DICOM patient folders
        ↓
RSNA DICOM Anonymizer
        ↓
De-identified DICOM
        ↓
pydicom metadata QC
        ↓
Visual QC using RadiAnt / OsiriX
        ↓
Correct series selected
        ↓
Research preprocessing
```

The raw identifiable hospital copy should be governed appropriately and not mixed with the research dataset.

---

6. RSNA DICOM Anonymizer

The purpose of RSNA anonymization is to remove or replace identifying DICOM information.

For example:

```
Original:

PatientName = XXXXXX
PatientID   = Hospital12345
```

```
After research de-identification:

PatientName = removed/replaced
PatientID   = SUBJ_0001
```

After anonymization, **pydicom should be used to verify that identifying metadata has actually been removed and that important imaging metadata remains usable**.

---

7. pydicom QC

pydicom is not being used as the AI model.

It is used to read and validate DICOM metadata.

For example, we may verify:

```
SeriesDescription
TransferSyntaxUID
Rows / Columns
PixelSpacing
SliceThickness
ImagePositionPatient
ImageOrientationPatient
StudyInstanceUID
SeriesInstanceUID
LossyImageCompression
```

For Module 2 it will also help verify that the correct DWI and ADC series have been exported.

---

8. Recommended research directory

It is better for labels to live in a CSV rather than relying only on folder names.

For Module 1, something like:

```
brain_mri_project/
│
├── module1_flair/
│   ├── M1_0001/
│   │   └── FLAIR/
│   │       ├── IMG0001.dcm
│   │       ├── IMG0002.dcm
│   │       └── ...
│   │
│   ├── M1_0002/
│   │   └── FLAIR/
│   │       └── ...
```

```
│   │
│   └── ...
│
├── module2_stroke/
│   ├── M2_0001/
│   │   ├── DWI/
│   │   │   ├── IMG0001.dcm
│   │   │   └── ...
│   │   └── ADC/
│   │       ├── IMG0001.dcm
│   │       └── ...
│   │
│   └── ...
│
└── integrated_test/
    ├── T_0001/
    │   ├── FLAIR/
    │   ├── DWI/
    │   └── ADC/
    └── ...
```

A Module 1 CSV could look like:

patient_id	flair_path	label	num_slices	plane	qc_status
M1_0001	module1_flair/M1_0001/FLAIR	Normal	25	Coronal	Pass
M1_0002	module1_flair/M1_0002/FLAIR	Abnormal	21	Coronal	Pass

Module 2:

patient_id	dwi_path	adc_path	label	dwi_slices	adc_slices	qc_status
M2_0001	.../DWI	.../ADC	Stroke	24	24	Pass
M2_0002	.../DWI	.../ADC	No_Stroke	26	26	Pass

---

9. DICOM → NIfTI

For actual preprocessing it will probably be easier to convert each complete DICOM series into a NIfTI volume using:

dcm2niix

Conceptually:

```
25 DICOM slices
       ↓
dcm2niix
       ↓
one .nii / .nii.gz volume
```

Keep the de-identified DICOM as the source archive.

NIfTI is simply more convenient for:

- volume manipulation
- masking
- cropping
- registration
- slice extraction
- NumPy/PyTorch processing.

---

10. Module 1 preprocessing

The proposed FLAIR preprocessing is approximately:

```
De-identified coronal FLAIR DICOM
                ↓
          dcm2niix
                ↓
         NIfTI volume
                ↓
           QC checks
                ↓
     Brain localization/masking
          using SynthStrip
                ↓
    Brain bounding box + margin
                ↓
      Remove low-brain slices
                ↓
     Intensity normalization
                ↓
 Resize while preserving aspect ratio
                ↓
          Pad to square
                ↓
 Grayscale → three identical channels
                ↓
            DINOv2
```

---

11. What is SynthStrip?

SynthStrip is a tool used for robust brain extraction/skull stripping.

It produces a **brain mask**.

Conceptually:

```
MRI
 ↓
SynthStrip
 ↓
Brain mask
```

```
0 = outside brain
1 = brain
```

Important implementation detail:

I would **not necessarily replace the original MRI with a skull-stripped image**.

Instead, use the mask primarily to determine:

Where is the brain?

Then calculate a bounding box around the mask and crop the **original MRI intensities** with a safety margin.

So:

```
Original MRI
      +
SynthStrip mask
      ↓
Find brain bounding box
      ↓
Expand box slightly
      ↓
Crop original MRI
```

This removes large amounts of useless black background while avoiding an excessively tight crop.

---

12. What about the first and last slices?

The beginning and end of many MRI stacks may contain:

- very little brain tissue,
- partial anatomy,
- edge effects,
- low useful information.

However, we should **not blindly delete exactly the first 3 and last 3 slices for every patient**.

Different patients have different:

- slice counts,
- anatomical coverage,
- positioning.

A better solution is to use the SynthStrip brain mask.

For every slice calculate something like:

brain coverage=brain-mask pixelstotal pixels\text{brain coverage} = \frac{\text{brain-mask pixels}} {\text{total pixels}}

Then exclude slices where brain coverage falls below a chosen threshold.

Conceptually:

```
Slice 1 → 1% brain   → remove
Slice 2 → 4% brain   → remove
Slice 3 → 18% brain  → keep
...
Slice 22 → 16% brain → keep
Slice 23 → 3% brain  → remove
```

The threshold should be chosen from data/QC rather than invented blindly.

So the idea is:

remove edge slices based on actual brain coverage, not fixed slice numbers.

---

13. Slight patient tilt

Some patients may lie slightly tilted in the scanner.

We should not automatically discard every mildly tilted scan.

DINOv2 and augmentation can tolerate modest variation.

Grossly abnormal orientation or severe acquisition problems should be flagged during QC.

We should avoid aggressive geometric correction unless genuinely necessary because unnecessary resampling can alter the image.

---

14. Intensity preprocessing

MRI intensity values do not have the same fixed physical scale as CT.

One scanner/patient may produce an image that looks darker or brighter than another.

Therefore preprocessing should normalize intensity.

A reasonable Module 1 strategy is:

```
Use pixels within brain mask
        ↓
Robust percentile clipping
        ↓
Brain-based z-score normalization
```

For example, extreme values can be clipped using percentiles before normalization.

N4 bias-field correction may be tested later as an **ablation**, rather than automatically making it mandatory.

---

15. Resize without distorting anatomy

Do not stretch every MRI directly into a square.

Instead:

```
Original crop
e.g. 210 × 180
       ↓
Resize longest dimension
       ↓
preserve aspect ratio
       ↓
pad remaining area
       ↓
224 × 224
```

This avoids changing anatomical proportions.

We can initially use **224 × 224**, since that works naturally with DINOv2-B/14, and perhaps later test 336 resolution if computationally feasible.

---

16. Why three channels if MRI is grayscale?

DINOv2 expects a 3-channel input because it was trained on ordinary RGB images.

MRI is grayscale.

We therefore use:

$X \rightarrow [X, X, X]$

so:

```
Channel R = MRI
Channel G = MRI
```

```
Channel B = MRI
```

No fake colors are added.

---

17. What exactly is DINOv2?

DINOv2 is a **self-supervised vision foundation model based on a Vision Transformer**.

It was originally trained on a huge collection of ordinary images.

It has learned general visual representations involving things like:

- shapes,
- edges,
- textures,
- structures,
- relationships between image regions.

We are not asking DINOv2 to diagnose an MRI directly.

We use it as a **feature extractor**.

For one MRI slice:

```
MRI slice
   ↓
DINOv2-B/14
   ↓
feature vector
```

If we use the CLS representation from DINOv2-Base:

slice→768-dimensional vector

So:

```
Slice 1 → [768 values]
Slice 2 → [768 values]
Slice 3 → [768 values]
...
```

Every slice gets the **same feature-vector dimension**.

---

18. Very important: we are NOT doing slice-level classification

We are doing:

slice-level feature extraction → patient-level classification

Not:

```
Slice 1 → Normal
Slice 2 → Normal
Slice 3 → Abnormal
```

Instead:

```
Slice 1 → 768-D feature
Slice 2 → 768-D feature
Slice 3 → 768-D feature
...
              ↓
        Aggregate features
              ↓
      one patient prediction
```

This distinction is important.

---

19. Different patients have different numbers of slices

This architecture naturally supports that.

Patient A:

25×76825\times768

Patient B:

21×76821\times768

Patient C:

28×76828\times768

The number of feature vectors varies.

The size of **each** feature vector remains fixed.

Then attention pooling converts the variable-length sequence into one fixed patient vector.

---

20. Why not mean pooling?

Mean pooling would simply calculate:

zpatient=1N∑i=1Nziz_{patient} = \frac{1}{N}\sum_{i=1}^{N} z_i

Every slice gets equal importance.

That may not be ideal because an abnormality might exist on only a few slices.

---

21. Why not majority voting?

Majority voting first makes a slice prediction:

```
Slice 1 → Normal
Slice 2 → Normal
Slice 3 → Abnormal
...
```

and then counts them.

The problem is that a small lesion may appear on only a few slices.

If:

```
20 slices = Normal
5 slices  = Abnormal
```

majority voting could potentially classify the whole study as normal.

It also reduces a rich 768-D representation to a binary label too early.

---

22. Attention pooling (The aggregation method to be used)

Attention pooling instead learns:

Which slice features matter most for the patient-level prediction?

Suppose:

```
Slice 1 → 0.02
Slice 2 → 0.05
```

```
Slice 3 → 0.55
Slice 4 → 0.30
Slice 5 → 0.08
```

These weights sum approximately to 1.

Then:

zpatient=∑iαiziz_{patient} = \sum_i \alpha_i z_i

The model therefore learns to give more importance to informative slices.

---

23. Why position-aware attention?

Ordinary slice attention knows:

what does the slice contain?

But MRI is an ordered anatomical stack.

We also want:

where approximately was this slice located in the stack?

So for slice ii among NN slices:

Pi = i/N-1

This maps position to approximately:

```
0 ---------------- 0.5 ---------------- 1
beginning          middle               end
```

This works even if:

```
Patient A = 21 slices
Patient B = 25 slices
```

because the position is **relative**, not a fixed slice number.

Attention can then use:

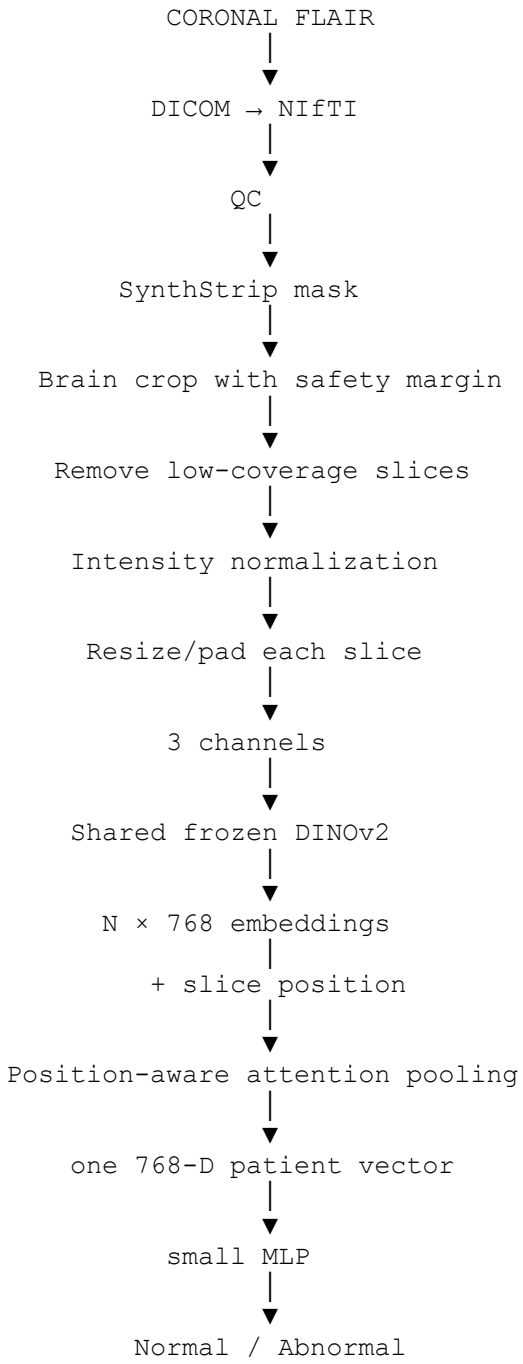
[zi,pi]

meaning:

what is visible + where it occurs.

24. Module 1 architecture

The basic implementation should therefore be:



Later:

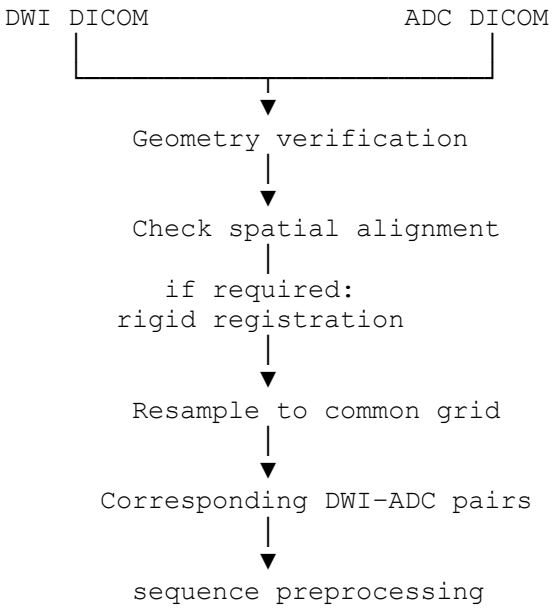
```
Normal / Abnormal
```

can replace the binary output once enough borderline cases exist.

25. Module 2 preprocessing

Module 2 is slightly different because DWI and ADC must correspond spatially.

The flow is:



Do **not** assume:

```
DWI IMG0001 = ADC IMG0001
```

just because filenames match.

Pairing should use DICOM spatial geometry such as:

```
ImagePositionPatient
ImageOrientationPatient
PixelSpacing
slice spacing
FrameOfReferenceUID
```

---

26. Different DWI and ADC resolution

Suppose:

```
DWI = 256 × 256
ADC = 512 × 512
```

If both cover the same anatomy, ADC can be resampled to the DWI reference grid.

Similarly, if slice thickness differs, alignment should be done in **physical space**, not by blindly pairing indexes.

ADC is quantitative, so preserve original DICOM/NIfTI values and avoid JPG conversion or arbitrary contrast manipulation.

---

27. Module 2 DINOv2 feature fusion

For each spatially matched pair:

```
DWI slice i → DINOv2 → 768-D
ADC slice i → DINOv2 → 768-D
```

Use the **same shared DINOv2 backbone**.

Then concatenate:

768+768=1536768 + 768 = 1536

and feed this into a small fusion MLP:

1536→7681536 \rightarrow 768

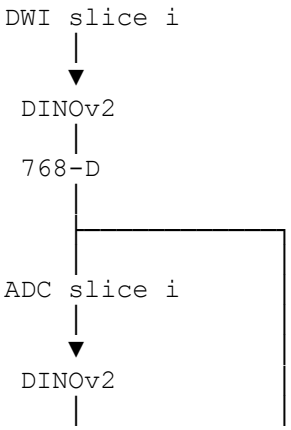
So:

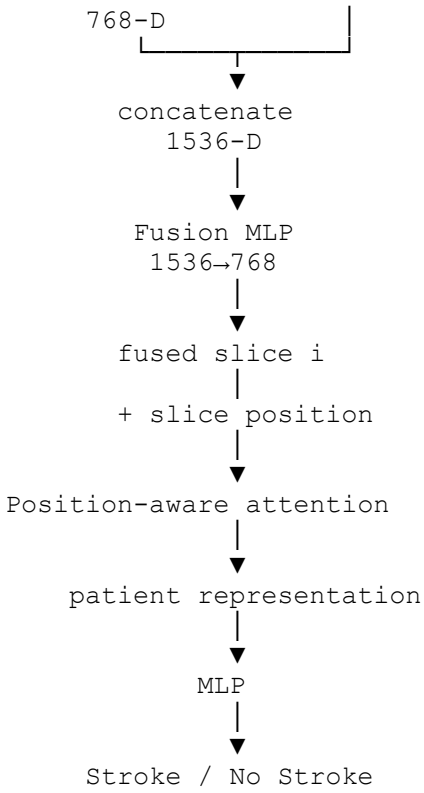
```
DWI feature ─┐
             ├── concatenate → Fusion MLP → fused slice vector
ADC feature ─┘
```

Then perform the same patient-level position-aware attention across the fused slice vectors.

---

28. Module 2 complete architecture





---

29. Training strategy

Initially, DINOv2 should be **frozen**.

Meaning:

```
DINOv2 weights
DO NOT update
```

Train only:

```
Module 1:
position-aware attention
classifier

Module 2:
fusion MLP
position-aware attention
classifier
```

This reduces:

- GPU requirement,
- training time,
- overfitting risk.

It also allows DINO embeddings to be precomputed once.

For example:

```
all MRI slices
     ↓
frozen DINOv2
     ↓
save embeddings to disk
```

Then the attention model trains on small vectors instead of repeatedly loading full MRI images.

Later, perform an ablation:

```
Frozen DINOv2
vs
last 2–4 transformer blocks fine-tuned
```

Full ViT fine-tuning is not the initial plan.

---

30. Data splitting

This must be done **patient-wise**. (all slices of a patient in training must be in training only, must not be in testing)

Wrong:

```
Patient A slice 1 → Train
Patient A slice 2 → Train
Patient A slice 18 → Test
```

Correct:

```
Patient A → Train
Patient B → Train
Patient C → Validation
Patient D → Test
```

All slices from one patient remain together.

This prevents leakage.

---

31. Separate datasets for the two modules

The modules do not need the same patients during training.

For example:

```
Module 1 training cohort
→ patients with usable coronal FLAIR

Module 2 training cohort
→ patients with usable paired DWI + ADC
```

They can be completely different patients.

Each module gets its own:

```
Train
Validation
Test/development evaluation
```

---

32. Integrated final validation/test data

For the final complete triage system, however, we need patients who have:

```
FLAIR
+
DWI
+
ADC
```

These patients must not have appeared in either module's training data.

Example:

```
integrated_test/
└── T_0001/
    ├── FLAIR/
    ├── DWI/
    └── ADC/
```

The CSV can contain:

patient_id	flair_label	stroke_label	expected_priority
T001	Normal	No Stroke	Routine
T002	Abnormal	No Stroke	Urgent
T003	Abnormal	Stroke	Immediate

---

33. Final decision fusion

Initial logic:

```
If Stroke = Yes
        ↓
     IMMEDIATE
```

```
Else
        ↓
look at Module 1

Normal
    ↓
ROUTINE

Abnormal
    ↓
URGENT
```

Later with Borderline:

```
Stroke positive → Immediate

No stroke + Abnormal → Urgent

No stroke + Borderline → Urgent / Priority Review

No stroke + Normal → Routine
```

Stroke overrides Module 1 because acute ischemic stroke is time-critical.

---

34. Explainability

There are two different explanation levels.

First:

Which slices influenced the patient-level result?

This comes from attention weights.

For example:

```
Slice 9  → 4%
Slice 10 → 7%
Slice 11 → 31%
Slice 12 → 39%
Slice 13 → 12%
```

Second:

Which region inside an important slice influenced DINOv2?

A ViT-compatible attribution/attention-rollout method can generate a heatmap.

Therefore the radiologist can potentially see:

```
Patient classified Abnormal
```

```
Most influential slices:
11, 12, 13

Regional heatmap:
highlighted suspicious region
```

The heatmap is **not a segmentation mask**.

---

35. Research gaps we are trying to address

A concise way to explain the methodological motivation is:

Existing limitation	Proposed response
Some systems classify slices independently	One patient-level classification
Mean pooling treats every slice equally	Learned attention weights
Majority voting throws away feature information	Aggregate rich DINO embeddings
2D models don't explicitly know slice location	Relative slice-position encoding
Fixed 3D CNNs often require fixed slice count	Variable-length slice aggregation
DWI-only stroke detection	Explicit DWI–ADC fusion
Many methods use conventional CNNs	Investigate DINOv2 foundation features
Tasks are studied separately	Combine abnormality + stroke into triage
Diagnostic output only	Routine/Urgent/Immediate worklist priority

Importantly, don't claim:

“DINOv2 + attention has never been done.”

DINOv2 slice aggregation already exists in the literature.

Our stronger research framing is the **combination** of:

```
general FLAIR abnormality
+
paired DWI–ADC stroke detection
+
variable-length patient-level aggregation
+
explicit slice position
+
final clinical triage fusion
```

---

36. Baselines and experiments

The implementation should not contain only one proposed model.

Useful comparisons are:

```
Module 1
CNN baseline
DINOv2 + mean pooling
DINOv2 + ordinary attention
DINOv2 + position-aware attention

Module 2
DWI only
DWI + ADC
mean pooling
position-aware attention

Training
Frozen DINOv2
vs
partial fine-tuning
```

This is what will tell us whether our individual architectural choices actually help.

---

37. Metrics

For both modules report:

```
Accuracy
Sensitivity / Recall
Specificity
Precision
F1-score
AUROC
Confusion matrix
```

For stroke especially, sensitivity matters strongly because false-negative stroke is the major under-triage risk.

A target such as:

≥95% stroke sensitivity

can be stated as an objective, not as a guaranteed result.

Thresholds should be selected on the **validation set**, not assumed to be 0.5.

The final test set remains untouched until final evaluation.

---

38. Practical software stack

A reasonable implementation stack is:

Purpose	Tool
De-identification	RSNA DICOM Anonymizer
DICOM reading/QC	pydicom
Visual DICOM review	RadiAnt / OsiriX
DICOM → NIfTI	dcm2niix
Brain extraction/localization	SynthStrip
NIfTI manipulation	nibabel
Image processing	NumPy / OpenCV / SciPy
Registration if required	SimpleITK / ANTs-style tooling
Deep learning	PyTorch
DINOv2	PyTorch / official model or compatible implementation
Metrics	scikit-learn
Tables/data	pandas

---

39. Implementation sequence I would give the person

The safest development order is:

Stage 1: verify and organize the DICOM dataset.

Stage 2: implement preprocessing and manually inspect outputs.

Stage 3: get Module 1 working with frozen DINOv2 + simple mean pooling first.

Stage 4: replace mean pooling with ordinary attention.

Stage 5: add relative slice position → position-aware attention.

Stage 6: develop Module 2 DWI/ADC pairing and simple fusion.

Stage 7: add patient-level attention to Module 2.

Stage 8: compare baselines and ablations.

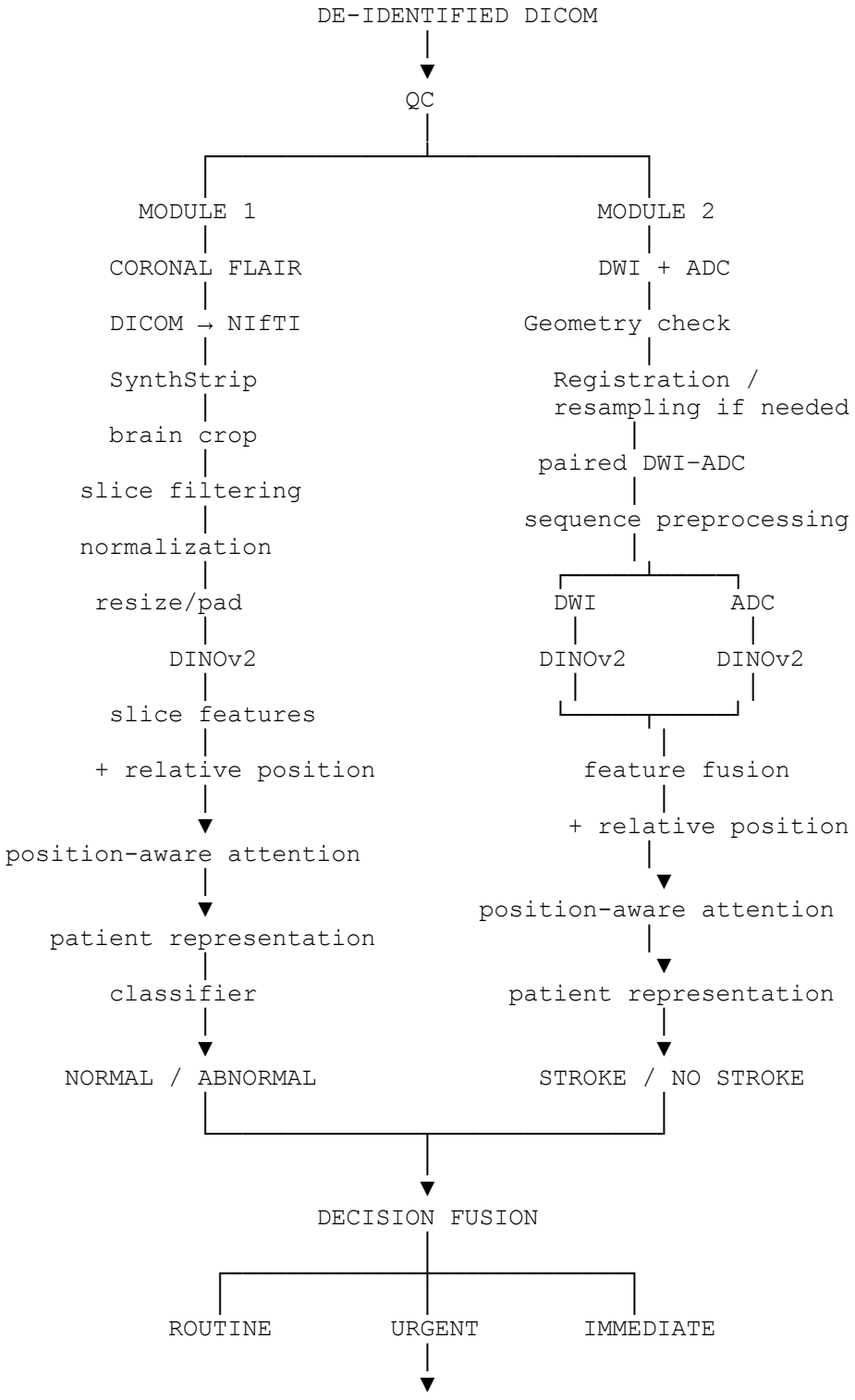
Stage 9: freeze both final modules and evaluate on integrated FLAIR+DWI+ADC patients.

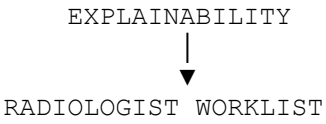
Stage 10: add decision fusion and explainability.

This incremental approach is much easier to debug than implementing everything at once.

---

40. The complete architecture in one diagram





:

DINOv2 is not doing slice-level classification. It is being used as a shared 2D feature encoder for every MRI slice; the patient-level prediction is produced only after those slice embeddings, together with their relative positions, are aggregated using attention.

And the second most important point is:

Do not hard-code preprocessing assumptions such as deleting the first and last three slices. Use image/brain-mask-based QC criteria wherever possible, because slice counts and anatomical coverage differ between patients.